

Movie Recommender System: Comprehensive Exploratory Data Analysis and Comparative Collaborative Filtering Evaluation

Amogh Roop Mandal(24125005) Ved Arya(24125038)

Abstract

This report outlines an end-to-end framework for large-scale movie recommendations. We first perform an exploratory data analysis (EDA) on a representative subsample of **19,363,053 explicit ratings**, tracking **331,337 unique users** and **16,532 movies**. Initial investigation reveals a **left-skewed rating distribution** (mean **3.602**, median **4.0**), alongside power-law long-tails governing both user activity and movie popularity profiles. Building on these characteristics, we execute a comparative evaluation of two core collaborative filtering paradigms: a latent factor Matrix Factorization framework optimized via **Singular Value Decomposition (SVD)** and a memory-based **K-Nearest Neighbors (KNN) Item-Based Collaborative Filtering** setup. Evaluated across predictive accuracy metrics (RMSE, MAE) and ranking performance parameters (MAP@10, NDCG, Recall, and Precision), **KNN Item-Based CF consistently outclasses SVD**, delivering an optimal RMSE of **0.8957**, a MAP@10 of **0.7277**, and expanded item catalog coverage (**33.5%**). Conversely, SVD reveals significant operational efficiency at runtime, showcasing an **11-fold prediction speedup (50.71 seconds)** compared to KNN (**560.29 seconds**). Diagnostics reveal a **regression-to-the-mean effect** in both models due to variance compression under explicit ratings imbalances. We discuss these technical trade-offs and propose a tiered hybrid deployment strategy optimized for low-latency web environments.

1 Introduction

Modern content-delivery platforms rely heavily on recommendation engines to address information overload and provide personalized discovery experiences. Scalable filtering strategies are broadly divided into **memory-based approaches**, which compute raw geometric similarity trends across item interaction vectors, and **model-based structural approaches**, which map user profiles and item characteristics directly onto a low-rank, shared latent vector space.

Evaluating these architectures requires a comprehensive approach. Traditional analytics frameworks, notably standard benchmarks like the historic Netflix Prize, prioritized pure rating prediction error minimiza-

tion. However, modern commercial ecosystems require recommenders to excel as **search and ranking frameworks**. Their primary real-world task is not guessing individual matrix cells, but rather structuring a high-quality top- N recommendation list.

This engineering report synthesizes our findings across two execution steps: (i) systemic data quality preprocessing and structural trait characterization via Exploratory Data Analysis; and (ii) a mathematical evaluation comparing baseline configurations of SVD Matrix Factorization against neighborhood KNN Item-Based Filtering pipelines across predictive error, top-10 list retrieval precision, runtime scaling profiles, and error distribution diagnostics.

2 Dataset Description and Preprocessing

2.1 Raw Dataset Allocation

The initial raw repository comprises explicit user-movie ratings spanning a 1-to-5 star integer spectrum. The complete source corpus tracking **≈ 100 million interactions** introduced out-of-memory (OOM) failures within the local compute infrastructure. To handle this constraint while preserving statistical integrity, a representative subset containing **20 million ratings** was pulled using uniform random sampling.

2.2 Sparsity and Quality Filtering

To address cold-start challenges and stabilize neighbor vectors, two sequential quality thresholds were applied to the working directory:

1. **Minimum User Activity Profile:** Users submitting **fewer than 10 ratings** across their historic timelines are pruned.
2. **Minimum Movie Coverage Profile:** Movies accumulating **fewer than 20 explicit ratings** are dropped.

Following preprocessing, the finalized working dataset tracks **19,363,053 clean interactions**, summarized in Table 1.

Table 1: Dataset descriptive statistics post-filtering.

Statistic Descriptor	Quantitative Value
Total Clean Ratings	19,363,053
Unique Registered Users	331,337
Unique Catalog Movies	16,532
Arithmetic Mean Rating	3.602
Median Rating Score	4.0
Standard Deviation (σ)	1.082
Global Rating Mode	4
Skewness Coefficient	-0.498

3 Exploratory Data Analysis (EDA)

3.1 Rating Density and Distribution Profile

Figure 1 visualizes the histogram tracking total ratings across the filtered repository. The distribution patterns display a clean *unimodal* shape governed by an evident *left-skew* ($\gamma_1 = -0.498$), with both the dataset mode and median **locking at 4.0 stars**.



Figure 1: Rating distribution and dataset summary statistics. The distribution is left-skewed (mode = median = 4 stars); 33.7% of ratings are four stars and 28.9% are three stars.

This negative skew implies that very low ratings (1–2 stars) are significantly underrepresented relative to their positive counterparts. This trend aligns with the phenomenon of *self-selection bias*, where users primarily rate movies they chose to watch, usually expecting a positive experience.

3.2 User Behavior and Activity Patterns

User interaction counts follow a **heavy-tailed power-law profile** (Figure 2, left). While the average interaction count scales to **58 ratings per profile**, this metric is highly inflated by an ultra-active cohort of power users, whereas the median user timeline records **only 35 interactions**.

Concurrently, evaluating active timeline density against rating tendencies (Figure 2, right) identifies a clear *variance compression* pattern. As individual user activity scales up, mean ratings converge tightly around the global mean (≈ 3.5 – 3.8 stars), showing that

high-volume rating behavior acts as a stabilizing regularizer.

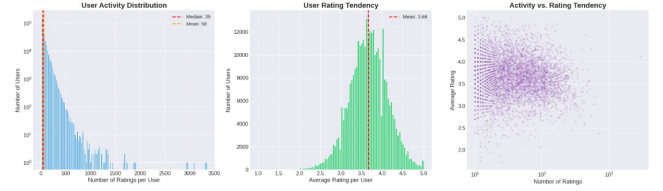


Figure 2: User behaviour: activity distribution (left) and variance compression scatter profile (right).

3.3 Movie Catalog Popularity Analysis

Movie visibility characteristics exhibit an extreme **heavy-tailed long-tail structure** (Figure 3). The median movie title captures **only 132 ratings**, while a highly concentrated, popular cohort dominates overall system traffic. Specifically, the **top 12.7% of movies** (2,101 individual titles) account for **80% of all ratings** in the system.

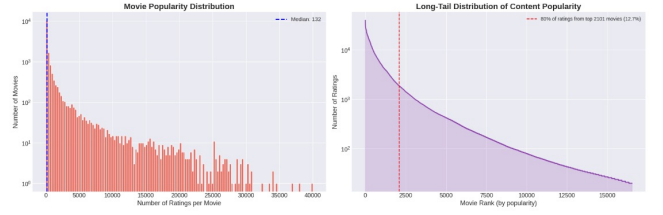


Figure 3: Movie popularity analysis showing long-tail distribution trends.

Evaluating the top 20 most frequently rated movies by volume (Figure 4) confirms that **high interaction volume does not necessarily imply high perceived quality**. Several high-volume titles exhibit average ratings matching or falling below the global baseline.

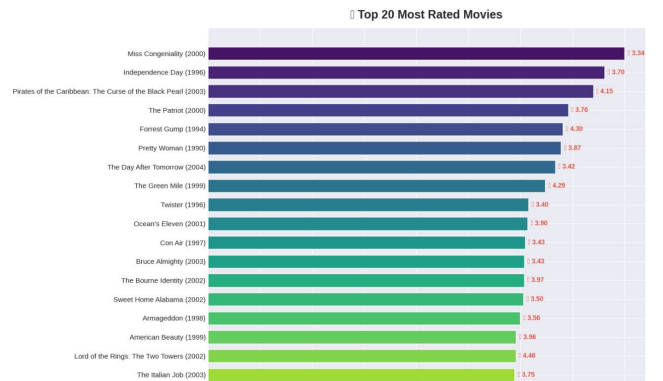


Figure 4: Top 20 most-rated movie titles sorted by total rating volume.

3.4 Temporal and Era Trends

Monthly rating frequencies from 2000 to 2006 (Figure 5) show a period of **rapid, near-exponential scaling** in

platform engagement beginning in 2002.

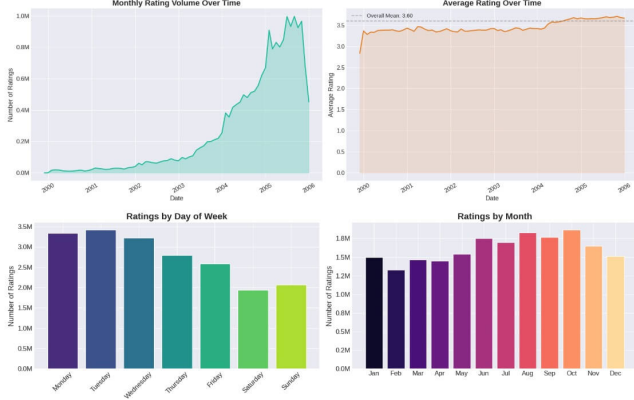


Figure 5: Temporal rating volume profiles computed across monthly aggregates.

Furthermore, analyzing historical release eras against average quality scores (Figure 6) highlights an apparent **downward trend in modern scores**. Pre-1940 films maintain higher average ratings, a pattern consistent with *survivorship bias*—only highly regarded classic titles are preserved and watched today, whereas contemporary releases exhibit ratings that regress toward the global mean.

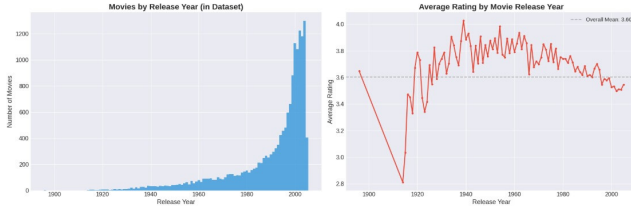


Figure 6: Movie release era tracking mean rating scores against theatrical release year.

4 Collaborative Filtering Algorithmic Frameworks

To build a personalized recommendation engine on top of these data insights, we implemented and compared two distinct collaborative filtering strategies.

4.1 Latent Factor Model via SVD

The SVD approach decomposes the sparse, observed rating matrix $R \in R^{U \times M}$ into low-rank user and item embeddings within a shared latent factor space of dimensionality d . The predicted rating $\hat{r}_{u,i}$ is modeled as:

$$\hat{r}_{u,i} = \mu + b_u + b_i + q_u^T p_i \quad (1)$$

where μ is the global baseline rating, b_u and b_i represent user and item bias deviations, and $p_u, q_i \in R^d$ define the latent embeddings. The optimization routine minimizes a regularized Mean Squared Error over observed ratings

$\mathcal{K}$:

$$\min_{b,p,q} \sum_{(u,i) \in \mathcal{K}} (r_{u,i} - \hat{r}_{u,i})^2 + \lambda (b_u^2 + b_i^2 + \|p_u\|_2^2 + \|q_i\|_2^2) \quad (2)$$

using Stochastic Gradient Descent (SGD) with learning rate γ .

4.2 Neighborhood Model via KNN Item-Based CF

The memory-based neighborhood technique maps relationships directly using item-to-item co-rating trajectories. Adjusted Cosine Similarity is utilized to isolate similarity scores while controlling for individual user scaling variances:

$$\text{sim}(i, j) = \frac{\sum_{u \in \mathcal{U}_{i,j}} (r_{u,i} - \bar{r}_u)(r_{u,j} - \bar{r}_u)}{\sqrt{\sum_{u \in \mathcal{U}_{i,j}} (r_{u,i} - \bar{r}_u)^2} \sqrt{\sum_{u \in \mathcal{U}_{i,j}} (r_{u,j} - \bar{r}_u)^2}} \quad (3)$$

where $\mathcal{U}_{i,j}$ marks users who evaluated both items, and $\bar{r}_u$ denotes the mean rating of user u . Unobserved ratings are predicted using a similarity-weighted average of the K closest neighbors to item i that user u has already rated ($\mathcal{N}_u(i)$):

$$\hat{r}_{u,i} = \frac{\sum_{j \in \mathcal{N}_u(i)} \text{sim}(i, j) \cdot r_{u,j}}{\sum_{j \in \mathcal{N}_u(i)} |\text{sim}(i, j)|} \quad (4)$$

5 Comparative Performance Evaluation

5.1 Predictive Error and Computational Profiling

Quantitative metrics tracking prediction accuracy and computational resource profiles are reported in Table 2.

Table 2: Model performance summary: predictive accuracy and execution timings.

Metric	SVD	KNN Item-CF	Winner
RMSE ↓	0.906388	0.895696	KNN
MAE ↓	0.705103	0.690902	KNN
Train Time (s) ↓	292.13	293.98	SVD
Predict Time (s) ↓	50.71	560.29	SVD

KNN Item-Based Collaborative Filtering achieves the highest prediction accuracy, reducing the mandatory RMSE target from **0.9064** down to **0.8957**, alongside a drop in MAE to **0.6909**. This demonstrates that localized item-to-item correlation graphs capture fine-grained rating behavior more effectively than the low-rank vector abstractions constructed by SVD.

However, analyzing system performance highlights a **major runtime trade-off**. While both configurations scale identically during training (**≈292–294 seconds**), their inference latencies diverge sharply. SVD executes test predictions in **50.71 seconds**, whereas KNN Item-CF requires **560.29 seconds**—representing an **11-fold**

increase in inference latency due to the cost of real-time neighbor lookups across large tables.

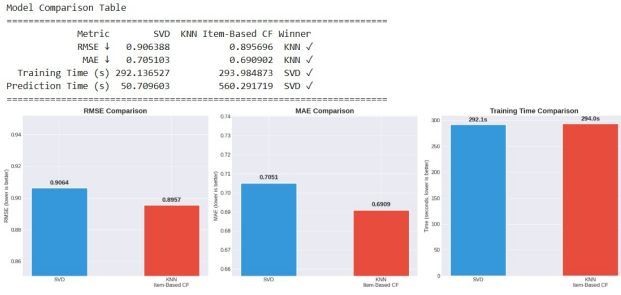


Figure 7: Predictive accuracy and performance benchmarks. Left: RMSE. Middle: MAE. Right: Operational training time profile.

5.2 Information Retrieval and List Ranking Quality

Evaluating the configurations as top-10 retrieval engines ($r_{u,i} \geq 4.0$ indicating true relevance) reveals consistent benefits for the neighborhood architecture (Table 3).

Table 3: Final performance summary across evaluation suites.

Metric	SVD	KNN Item-CF
RMSE (Mandatory) ↓	0.9064	0.8957
MAE ↓	0.7051	0.6909
MAP@10 (Mandatory) ↑	0.7071	0.7277
Precision@10 ↑	0.6933	0.7043
Recall@10 ↑	0.6808	0.6906
NDCG@10 ↑	0.8149	0.8305
Hit Rate@10 ↑	0.9990	0.9994
Training Time (s) ↓	292.1	294.0
Catalog Coverage	32.2%	33.5%
(Out of 16,532 movies)	(5,329 titles)	(5,540 titles)

The mandatory ranking target, **MAP@10**, increases from **0.7071 up to 0.7277** when switching to KNN Item-CF. This advantage is maintained across all ranking dimensions: **NDCG@10** increases to **0.8305** (implying better ranking ordering), while **catalog coverage expands to 33.5%** (5,540 movies recommended). SVD’s lower coverage indicates a tendency to concentrate recommendations on dominant global hits, whereas KNN effectively surfaces localized, long-tail options.

6 Residual Diagnostics and Variance Compression

To explore systematic errors, Figure 9 plots the residual distribution and prediction errors for the SVD model.

The error distribution matches a standard bell curve centered near zero, confirming proper convergence. However, the actual vs. predicted scatter matrix (Figure 9, right) reveals a notable **variance compression effect**:



Figure 8: Left: Evaluation across ranking parameters. Right: Visual trade-off mapping pure prediction error (RMSE) directly against top-10 list ranking precision (MAP@10).

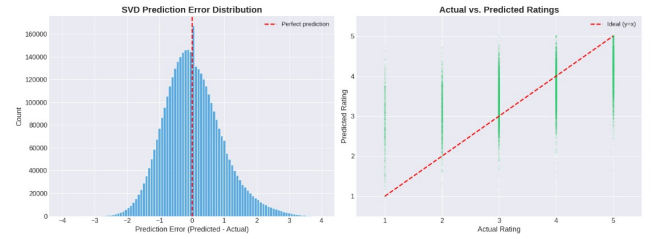


Figure 9: SVD predictive diagnostics. Left: Residual error distribution ($e = \hat{r} - r$). Right: Actual versus predicted rating scatter plot highlighting variance compression.

- **Low ground-truth values** (1–2 stars) are systematically **overpredicted**, clustering between 2.5 and 4.0.
- **High ground-truth scores** (5 stars) are **suppressed**, dropping down toward the 3.5 to 4.5 sector.

This behavior demonstrates a **regression-to-the-mean effect** driven by the dataset’s left-skewed distribution (identified during the EDA phase). Because the loss function penalizes large squared errors, the model learns a conservative strategy: it avoids predicting extreme values and pulls its estimates toward the global mean (≈ 3.60). Consequently, the system overpredicts low-quality movies and underpredicts high-quality titles.

7 Deployment Strategy and Operational Discussion

Selecting the optimal model for production deployment involves navigating a clear trade-off between **accuracy and latency**. While KNN Item-CF delivers **superior list precision** (MAP@10 = 0.7277) and broader catalog exploration, its **high inference latency** (560.29 seconds) makes it impractical for real-time recommendation generation in live production environments. SVD Matrix Factorization operates with **exceptional speed**, generating predictions 11 times faster than the KNN model, but at the cost of slightly lower ranking accuracy.

To balance these strengths, a commercial movie plat-

form should adopt a **tiered hybrid architecture**:

- **Offline Batch Layer (KNN)**: Compute personalized top-10 recommendation vectors for active users during off-peak hours using the accurate KNN model, and cache the results in a low-latency key-value store (e.g., Redis).
- **Online Real-Time Layer (SVD)**: For active sessions, or when a user’s cached profile is invalidated by new interactions, fall back to the SVD model. Because SVD models store user and item factors as low-rank embeddings, they compute instant recommendations on the fly using **fast vector dot products**.

8 Conclusion

This report analyzed a dataset of **20 million movie ratings**. Our EDA revealed a **left-skewed rating distribution** (median = 4.0) and heavy-tailed long-tails governing user and movie engagement patterns. In the modeling phase, **KNN Item-Based Collaborative Filtering achieved the highest predictive and ranking accuracy** (MAP@10 = 0.7277), while **SVD Matrix Factorization demonstrated clear operational advantages** with an **11-fold reduction in inference latency**. Both algorithms exhibit a variance compression bias that pulls extreme values toward the global mean. Future development will focus on addressing this bias by implementing weighted cross-entropy loss functions, exploring deep neural collaborative filtering (NCF) techniques, and introducing temporal dynamics to track shifting user preferences over time.

9 Team Contributions

The workload for this study was distributed as follows:

Table 4: Summary of team contributions.

Team Member	Primary Contributions
Amogh Roop Mandal	KNN Item-Based CF model training, metrics evaluation (ranking and retrieval), data preparation for modelling, and cold start strategy formulation.
Ved Arya	SVD Matrix Factorization model training, comprehensive Exploratory Data Analysis (EDA), dataset sampling and filtering, and temporal trend analysis.

References

- [1] F. M. Harper and J. A. Konstan, “The MovieLens Datasets: History and Context,” *ACM Trans. Interact. Intell. Syst.*, vol. 5, no. 4, pp. 1–19, 2015.
- [2] Y. Koren, “Collaborative Filtering with Temporal Dynamics,” in *Proc. ACM KDD*, 2009, pp. 447–456.
- [3] Y. Koren, R. Bell, and C. Volinsky, “Matrix Factorization Techniques for Recommender Systems,” *IEEE Computer*, vol. 42, no. 8, pp. 30–37, 2009.
- [4] X. He et al., “Neural Collaborative Filtering,” in *Proc. WWW*, 2017, pp. 173–182.
- [5] Y.-J. Park and A. Tuzhilin, “The Long Tail of Recommender Systems and How to Leverage It,” in *Proc. ACM RecSys*, 2008, pp. 11–18.